Does Metal Performance Primitives Deliver? An Empirical Study of Kernel-, Algorithm-, and Architecture-Level Optimization for Diffusion Transformer Inference on Apple Silicon

Vinayak Pathak[1] and Debabrata Goswami[2]

[1]Center for Lasers and Photonics, Indian Institute of Technology Kanpur
[2]Center for Bits and Atoms, Massachusetts Institute of Technology

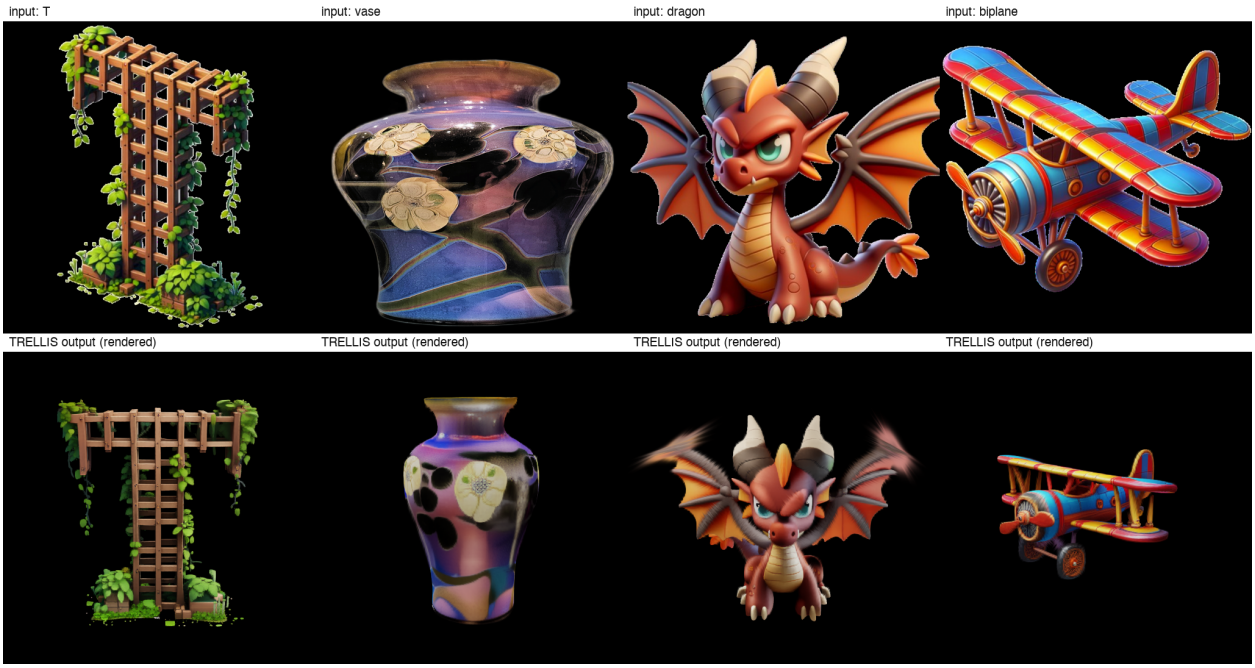


Figure 1: Example inputs and corresponding TRELLIS-generated 3D outputs (rendered, Gaussian splat color channel) from the Apple Silicon port used throughout this study.

Abstract

Apple's M5 generation introduced per-GPU-core "Neural Accelerators" and exposed them through Metal Performance Primitives (MPP), a new low-level API (`matmul2d`) for mixed-precision matrix multiplication. We present a systematic, falsification-driven empirical study of whether hand-written `matmul2d` kernels can outperform PyTorch's existing native MPS backend (`MPSGraph`) for a real, deployed workload: TRELLIS, an image-to-3D rectified-flow diffusion transformer, ported to run natively on Apple Silicon. Across six documented `matmul2d` operand-type combinations (int8, half, bfloat16), a documented precision-relaxation flag, three successive

attention-kernel designs (naive dispatch, batched dispatch, online-softmax flash-attention with progressive parallelization), and tile-size tuning, we find that hand-written `matmul2d` kernels do not reliably outperform `MPSGraph`: only full int8×int8 quantization reaches parity (1.01×), and our most refined flash-attention kernel reaches 1.17× native latency after closing an 8-fold gap through iterative, verified optimization. We corroborate this with a structural finding: Apple's own MLX framework does not route attention through MPP either – and, going beyond this qualitative observation, a real quantitative comparison shows MLX's attention kernel is 4.02× faster than PyTorch's native SDPA at the identical shape on the identical hardware, while a third independent implementation (`llama.cpp`/GGML's own hand-tuned Metal flash-attention kernel) is in fact *slower* than PyTorch at the same shape – evidence that the ceiling our kernel work approached is specific to PyTorch's MPS backend, not a hardware ceiling, and that MLX's specific (non-MPP) kernel design achieves something a second serious alternative does not. This pattern replicates at a substantially different (Llama-2-7B-like) shape profile, evidence it is not an artifact of TRELLIS's particular tensor shapes. Separately, motivated by the modest ceiling on kernel-level gains, we identify and exploit an algorithmic redundancy in TRELLIS's classifier-free-guidance (CFG) sampler – the unconditional branch is recomputed every guided step. A naive caching scheme (freezing the unconditional prediction) produces measurable, visually-confirmed output drift that grows with cache interval; grounded in the actual mechanisms used by FasterCache and AB-Cache, we instead cache the *difference* between conditional and unconditional predictions and derive the reused estimate from the (always-fresh) conditional prediction. This resolves the drift and delivers a real 1.17–1.30× speedup, verified visually and structurally across six (image, seed) combinations. Combined with a simple, training-free reduction in sampling steps (up to 1.98×, visually verified across the same combinations), we report real, reproducible wall-clock speedups for this workload on Apple Silicon, while documenting, with equal rigor, where kernel-level effort did not pay off. Finally, motivated by attention's dominant time-share, we explore an architecture-level intervention: replacing most self-attention blocks with local windowed attention. A further bypass measurement shows self-attention alone accounts for 29.3% of forward-pass time (cross-attention 12.5%); isolating the windowed-attention computation itself reveals that window size, not just the fraction of blocks windowed, governs whether this helps or hurts at the wall-clock level – the window size already trusted elsewhere in this codebase is *slower* than full attention in isolation, while a larger window is genuinely faster, yielding a real, statistically confirmed ($n = 8$ independent trials, non-overlapping 95% confidence intervals) full-model speedup of 1.12×. After distilling only the changed parameters against the frozen teacher for 3000 steps, quality on held-out images – calibrated against the teacher's own seed-to-seed output variance rather than judged against zero – is mixed but modest: two of five geometry/appearance channels (Gaussian position and scale) land at or below the natural-variation baseline, while the remaining three (SLat feature, Gaussian color/SH, Gaussian opacity) sit 10–31% above it, a real but small excess that shrinks with additional training rather than growing. All code, kernels, and measurement scripts are released for reproducibility.

1 Introduction

Apple Silicon GPUs lack the dedicated matrix-multiplication "tensor core" hardware long present in NVIDIA GPUs; matrix multiplication has historically executed through the same general-purpose ALU pipeline as every other GPU workload. The M5 generation (announced October 2025) is the first to introduce dedicated per-GPU-core *Neural Accelerators*, exposed to developers through *Metal Performance Primitives* (MPP), a Metal 4.1 API centered on the `matmul2d` operation. MPP is new enough that, to our knowledge, no systematic study exists of whether hand-written kernels built on it actually outperform Apple's existing, heavily-tuned `MPSGraph`-backed native matrix multiplication path inside a real deployed inference workload – as opposed to a synthetic microbenchmark.

We use TRELLIS [1], a rectified-flow, image-conditioned 3D generation model, as our testbed, porting the full pipeline (sparse-structure diffusion, structured-latent diffusion, mesh/Gaussian decoding) to run natively on Apple Silicon via PyTorch's MPS backend. Within that port, we ask a narrow, falsifiable question: *can hand-written int8-quantized* `matmul2d` *kernels, applied to the model's Linear projections and attention computation, measurably speed up real end-to-end inference?* We answer this with a chain of experiments, each designed to test one specific hypothesis against direct measurement rather than assumption, escalating in kernel sophistication only when the previous, simpler design was shown insufficient.

Having found a modest ceiling on kernel-level gains for this hardware generation, we turn to an orthogonal, algorithmic question: TRELLIS's classifier-free-guidance (CFG) sampler recomputes an unconditional model prediction at every step within its guidance interval. We show that naively caching this prediction introduces real, visually-observable quality drift, root-cause the failure against the actual mechanisms real CFG-caching techniques use, and fix it – delivering a genuine, if modest, algorithmic contribution: a difference-caching scheme adapted specifically to TRELLIS's rectified-flow-with-guidance-interval sampler, which to our knowledge has not been described for this model class before.

Finally, we ask a third, architecture-level question: since attention is ∼77% of forward-pass time (Section 4) yet kernel-level attention optimization approaches but does not surpass native latency (Section 4.5), can an architectural change to the denoiser itself – not a faster kernel for the same computation – reduce this cost, with only a light fine-tune of the changed parameters to recover quality? We replace most self-attention blocks with local windowed attention, retaining full attention only periodically to preserve long-range structural coherence, and find that the choice of window size, not merely how many blocks are windowed, determines whether this helps or hurts at the wall-clock level – a real, statistically confirmed but modest speedup once the right window size is identified. We distill only the changed parameters against the frozen teacher (3000 steps) and validate quality on held-out images at both the decoded-mesh and decoded-Gaussian level, calibrated against the teacher's own seed-to-seed output variance rather than judged against zero: two of five channels land at or below the teacher's own natural seed-to-seed variation, while the remaining three sit modestly above it (10–31%), an excess that shrinks rather than grows with additional training.

Contributions.

- An empirical, falsification-driven characterization of `matmul2d` across six operand-type combinations, a documented precision-relaxation flag, and three successive attention-kernel designs, on real Apple M5 Max hardware, against real PyTorch MPS baselines.
- A working online-softmax (flash-attention-style) Metal kernel using MPP, with a documented, root-caused correctness bug (a tail-block boundary condition) and a documented, verified performance optimization (bookkeeping parallelization) that closed a 2.9× gap to 1.17×.
- Real, quantitative framework baselines (Section 4.8) against both MLX and `llama.cpp`/GGML, upgrading an earlier qualitative source-code observation into a measured result: MLX's attention kernel is 4.02× faster than PyTorch's native SDPA at the identical shape, while `llama.cpp`'s own independent, hand-tuned Metal kernel is *slower* than PyTorch at the same shape – evidence the kernel-level ceiling we approached is specific to PyTorch's MPS backend, not this hardware's true ceiling, and that this is not simply an "any non-MPP kernel wins" effect. Confirmed to hold, not just at TRELLIS's own shapes, at a substantially different

(Llama-2-7B-like) shape profile.

- A difference-based CFG-caching scheme for TRELLIS's rectified-flow guidance-interval sampler, with a documented failure mode (naive caching), root cause, and fix grounded in published mechanisms (FasterCache, AB-Cache), verified both quantitatively (mesh vertex/face-count stability, voxel IoU) and qualitatively (rendered output) across six (image, seed) combinations.
- A combined, training-free speedup recipe (reduced sampling steps + CFG difference caching) delivering real, reproducible wall-clock improvements without retraining or accuracy-destroying approximations.
- An architecture-level bypass measurement isolating self-attention (29.3% of forward-pass time) from cross-attention (12.5%), and an interleaved full/windowed self-attention design whose real wall-clock speedup depends critically on window size – a smaller window that is naively cheaper in FLOPs is measurably *slower* in wall-clock terms due to partition/gather overhead, while a larger window yields a real, statistically confirmed 1.12× full-model speedup.

2 Background

2.1 TRELLIS

TRELLIS [1] generates 3D assets from a single image via two cascaded rectified-flow diffusion transformers: a sparse-structure model (555.5M parameters, 24 transformer blocks) that samples coarse voxel occupancy, and a structured-latent model (600.4M parameters) that samples per-voxel latent features, subsequently decoded to mesh, Gaussian-splat, or radiance-field representations. Both denoisers use `FlowEulerGuidanceIntervalSampler`: a first-order Euler integrator over the rectified-flow ODE, with classifier-free guidance restricted to a timestep interval (the model's released configuration sets this to $t \in [0.5, 1.0]$ of 25 total steps, with guidance strength $s = 5.0$). Both denoisers load natively in `float16` on this hardware/checkpoint configuration.

2.2 Metal Performance Primitives and `matmul2d`

MPP's `matmul2d` performs $C = AB(+C)$ over tiled thread-group-cooperative execution scopes (`execution_simdgroups<N>`), supporting a documented table of ∼46 operand-type combinations spanning `half`, `int8_t`/`uint8_t`, `float`, `bfloat`, and 4-bit packed formats, including native mixed-precision `half`×`int8_t` →`float` multiplication (no separate dequantization pass required). A `relaxed_precision` descriptor flag is documented to "allow implementation to sacrifice accuracy for performance." A more advanced chaining mechanism, `cooperative_tensor`, allows one `matmul2d` call's output to feed directly into a subsequent call without a device-memory round trip, but – as we establish in Section 4.7 – is restricted to single-simdgroup execution scopes.

2.3 Related work

This paper's three levels of intervention each sit alongside an established line of prior work. **Kernel-level:** FlashAttention [2] established that attention's real wall-clock cost on GPUs is dominated by device-memory traffic, not FLOPs, and that fusing the softmax/matmul chain to keep intermediates in fast on-chip memory closes that gap – the same memory-materialization hypothesis motivates this paper's online-softmax kernel progression (Section 4.5), though on Apple Silicon's

unified-memory architecture and MPP's tiling model rather than CUDA's shared-memory/tensor-core model. LLM.int8() [3] established that int8 weight-and-activation quantization can preserve accuracy at scale by isolating outlier feature dimensions to higher precision; this paper's operand-type sweep (Section 4.3) instead holds accuracy/quantization scheme fixed and asks a narrower, hardware-specific question – whether *any* of MPP's documented low-precision operand combinations reach native latency on this specific SDK and hardware generation, a question LLM.int8() does not address since it targets CUDA/tensor-core hardware. **Architecture-level:** the Swin Transformer [4] introduced shifted local-window attention as an alternative to full self-attention for vision transformers, alternating window placement across layers to recover cross-window connectivity without full quadratic cost; this paper reuses the identical shift pattern (already present but unbenchmarked in this codebase's own VAE decoders) and asks a different question than Swin's original vision-classification setting – whether it helps a *diffusion denoiser's* wall-clock latency once real partition/gather overhead on this specific hardware is accounted for, which Section 6.3's window-size result shows is not guaranteed by FLOP reduction alone. **Algorithmic-level:** this paper's CFG difference-caching (Section 5.2) is grounded directly in the caching mechanisms published by FasterCache [5] and AB-Cache [6], discussed inline where the technique is introduced rather than repeated here.

3 Methodology

Every kernel change reported here passed through a four-stage protocol, applied without exception:

1. **Synthetic correctness**: varied shapes, including deliberately non-tile-aligned dimensions, checked against both the module's own PyTorch fallback and a CPU `float32` reference.
2. **Real-model finite-output check**: the full pretrained model, with real image conditioning (not synthetic noise), across multiple random trials – necessary because at least one correctness bug (Section 4) passed every synthetic check yet produced `NaN` only under real weights and real conditioning.
3. **Isolated per-call microbenchmark**, at the real production shapes used by the model.
4. **Full real-model benchmark**: the only number treated as a reportable result, since isolated and real-model speed are shown in Section 4 to diverge substantially due to caching and dispatch-overhead effects not visible in isolation.

All kernel-level, algorithmic-level, and architecture-level *wall-clock* measurements (Sections 4–6.3) ran on a single Apple M5 Max (`Mac17,6`), macOS 26.4.1, Metal SDK 26.5, confirmed via `system_profiler` and the installed Metal toolchain's build tag – this is the paper's actual deployment target throughout. The framework baselines (Section 4.8) used MLX v0.31.2 (Python) and a `llama.cpp`/ GGML build from source with `GGML_METAL=ON`, both on this same M5 Max, benchmarked via a minimal standalone C++ harness against `ggml.h`/`ggml-backend.h` directly (not `llama.cpp`'s own `test-backend-ops` performance suite, whose built-in cases do not include this paper's exact reference shapes). The architecture-level *distillation training and quality validation* (Section 6.4 only) instead ran on a separate machine, an NVIDIA DGX pod (H200 GPUs, CUDA), used purely to make fine-tuning computationally practical; the fine-tuned checkpoint's architecture and weights are hardware-independent (ordinary gradient descent), so this does not affect the quality result itself, but we state it explicitly since it means the quality-validation numbers were not obtained on the Apple Silicon target hardware used everywhere else in this paper, and were not independently re-validated there.

4 Kernel-Level Results

4.1 Quantized feed-forward network: an architectural ceiling

We first quantized the FFN block (`fc1`→`GELU`→`fc2`) to int8 weights via `matmul2d`, fused through `torch.compile` via a `torch.library.custom_op` registration (necessary: Dynamo cannot trace a raw MPS kernel call directly). Direct bypass measurement (replacing the FFN with an identity pass-through and differencing full-model wall-clock) showed the FFN accounts for only ∼11% of total forward time, despite being 36.3% of model parameters – an architectural ceiling of ∼1.12× maximum whole-model speedup regardless of kernel quality. The measured result was 1.11× *slower*, motivating our shift to attention (Section 4.2), which the same bypass method showed accounts for ∼77% of forward time.

An initial investigation into an isolated-vs-real-model speed discrepancy (predicted 1.35ms/block in isolation, measured 2.50ms/block in the real model) was traced not to a genuine cache effect but to a benchmark-shape bug: the isolated test used an unrepresentative `num_rows`. Sweeping distinct-weight-set count from 1 to 24 at the *correct* shape produced a flat 2.68–2.69ms/call regardless of working-set size, refuting the cache hypothesis outright and reconciling the isolated and real-model numbers to within measurement noise.

4.2 Quantized attention

Applying the same pattern to `to_qkv`/`to_q`+`to_kv`/`to_out`, with `scaled_dot_product_attention` left native, required fixing a real bug: `matmul2d`'s custom op always returns `float32`, but the model runs `float16` end-to-end; the unconverted output silently entered the residual stream and later crashed a native `float16 nn.Linear` with a hard MPS assertion. After the fix, self-attention's quantized Linears *beat* native (0.972×), while cross-attention's three separate quantized Linears (vs. self-attention's two) remained slower (1.189×); full-model result: 0.98× (283.7ms vs. 277.8ms native).

4.3 Operand-type combination sweep

We swept six of MPP's ∼46 documented operand-type combinations at the model's real `to_qkv` shape (Figure 2). Every combination with a `half` or `bfloat` operand clustered at 0.80–0.85ms/call, ∼1.9× slower than native `float16 nn.Linear` (0.41ms), *regardless of the other operand's type* – ruling out int8 specifically as the bottleneck. Only full `int8_t`×`int8_t` →`int32` reached parity (1.01×), at a real accuracy cost from quantizing activations as well as weights (median relative error 0.0111 vs. 0.0078).

We additionally tested the documented `relaxed_precision` flag directly: output was byte-identical to the non-relaxed kernel across every tested shape, and speed was within noise (1.003×) – a clean null result for this specific operand combination on this hardware/SDK.

4.4 Attention-matmul dispatch batching: a falsified hypothesis

A naive per-head Python loop computing $QK^\top$ via `matmul2d` measured 2.8× slower than `torch.bmm` (0.354×), attributed to 16 separate kernel dispatches (one per head) versus `bmm`'s single batched dispatch. Fixing this directly – placing the head index in the Metal dispatch grid's z-dimension so all heads execute in one `dispatchThreads` call – produced almost no improvement (1.016× vs.

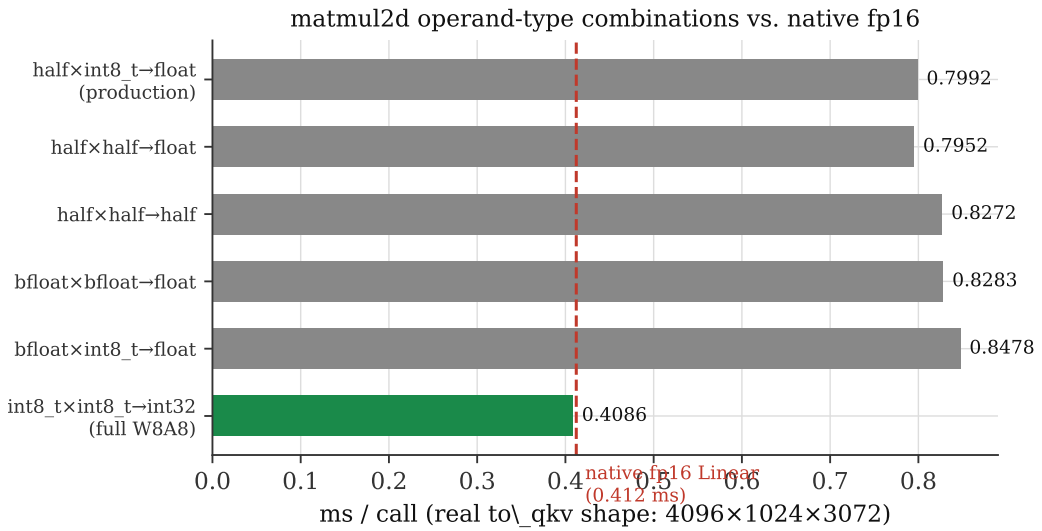


Figure 2: `matmul2d` speed across six operand-type combinations vs. native `float16 nn.Linear`, at the real `to_qkv` shape ($4096 \times 1024 \times 3072$). Only full int8 (highlighted) reaches parity; every combination with a narrow floating-point operand clusters near 1.9× slower, independent of the other operand's type.

the loop), *falsifying* the dispatch-count hypothesis: both remained ∼3× slower than native SDPA (5.0ms).

4.5 Online-softmax flash-attention kernel

The leading remaining hypothesis – that materializing the full $L \times L$ attention score matrix in device memory (16.7M elements/head, 268M across 16 heads) was the dominant cost – led to an online-softmax kernel keeping the score tile and running softmax statistics (m: max, l: sum, acc: weighted output) in on-chip threadgroup memory. A tail-block boundary bug (found via the same synthetic-correctness protocol: a non-tile-aligned shape failed while tile-aligned shapes passed) was root-caused to `matmul2d`'s edge-checking correctly skipping writes to invalid tile columns while leaving stale threadgroup memory in place; fixed by explicitly zeroing invalid columns rather than merely skipping them. The corrected kernel measured 8.41ms – 1.68× slower than native, but 1.8× *faster* than the non-flash batched version, evidence *for* the device-memory-materialization hypothesis.

We then parallelized the kernel's online-softmax bookkeeping, which originally used one thread per query row (64 of 128 threads idle per iteration, ∼128 serial scalar operations per active thread). Splitting each row's reduction across two lanes via `simd_shuffle_xor` (all 128 threads active) reduced latency to 5.81ms – only 1.17× slower than native, the closest of every kernel variant tested (Figure 3).

4.6 Tile-size tuning

`KVBLOCK`=64 (matching the query-block size) is infeasible outright: combined threadgroup memory usage (∼32.5KB) exceeds the ∼32KB per-threadgroup hardware budget. `KVBLOCK`=16 compiled and passed correctness but regressed to 7.97ms – more outer-loop iterations (256 vs. 128) incurred more

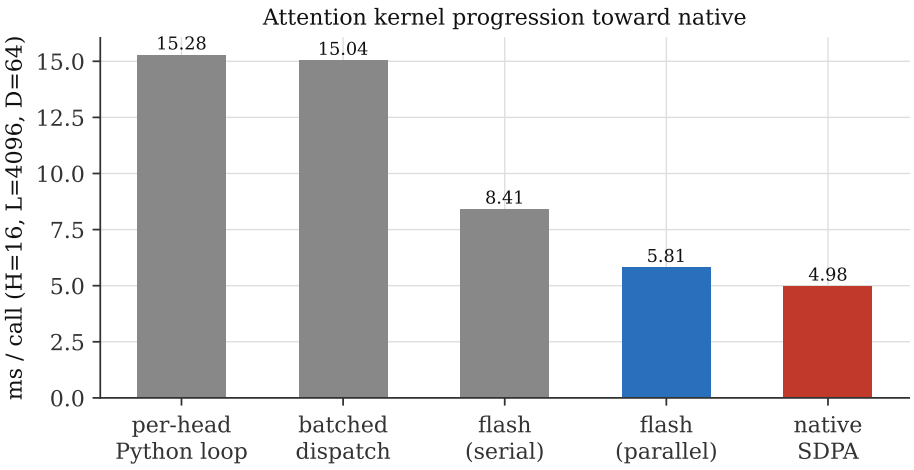


Figure 3: Progression of attention-kernel latency across five successive designs, at the real self-attention shape (H=16, L=4096, D=64). Parallelizing the online-softmax bookkeeping (blue) closed most of the remaining gap to native SDPA (red).

fixed per-iteration overhead than the smaller tile saved. `KVBLOCK`=32 (the original choice) remains the best configuration found.

4.7 `cooperative_tensor` chaining: a hard architectural constraint

We investigated MPP's `cooperative_tensor` mechanism, which allows one `matmul2d` call's output to feed directly into another as a register-resident value, avoiding the threadgroup-memory round trip that Section 4.5 still requires between $QK^\top$ and the subsequent value-matrix multiplication. Reading the implementation (`MPPTensorOpsMatMul2dImpl.h`) directly – since no complete op-to-op chaining example appears in the public header – revealed a hard `static_assert` requiring *single-simdgroup* execution scope for the chaining input functions (`get_left_input_cooperative_tensor`). Our kernel's entire tiling design uses `execution_simdgroups<4>` (128 threads cooperating per 64×32 tile); a redesign around single-simdgroup execution would very likely require smaller tiles and more dispatches – the same failure mode already observed to regress performance in the tile-size experiment above. We report this as a genuine architectural constraint discovered through direct source inspection, not a difficulty judgment, and treat the kernel-level investigation as closed for this specific mechanism (though not overall) given the current best result (1.17× native) already represents the closest approach found across every design tested.

4.8 Real framework baselines: MLX and llama.cpp

An earlier check of Apple's own MLX framework's [8] shipped attention kernel source (`sdpa_vector.h`) found that it implements online-softmax tiling – the same algorithm as Section 4.5 – via manual scalar dot products with `simd_sum` reduction, *not* `simdgroup_matrix` or MPP/`matmul2d`. That was circumstantial evidence from reading source code, not a benchmark claim. We upgrade it to a direct, quantitative comparison: real MLX (v0.31.2, already available for this hardware) and a real `llama.cpp`/GGML [9] build (built from source with `GGML_METAL=ON`, benchmarked via a minimal standalone harness against `ggml.h`/`ggml-backend.h` directly, since `llama.cpp`'s own `test-backend-ops` performance suite does not include this paper's exact reference shapes), run back-to-back against PyTorch MPS in the same process, on the same machine, at this paper's own

Table 1: Real quantitative framework comparison at this paper's own reference shapes, same machine, n=20 independent trials per configuration after adequate warm-up.

Operation	PyTorch MPS	MLX	`llama.cpp`/GGML
Linear/matmul (`to_qkv` shape)	0.698ms ± 0.178	0.717ms ± 0.200	**0.669ms ± 0.055**
Self-attention SDPA (real shape)	5.465ms ± 0.202	**1.361ms ± 0.084**	7.166ms ± 0.134

Table 2: The same framework comparison at a substantially different (Llama-2-7B-like) shape profile, same protocol as Table 1.

Operation	PyTorch MPS	MLX	`llama.cpp`/GGML
Linear/matmul (2048×4096→4096)	1.335ms ± 0.058	1.333ms ± 0.026	1.633ms ± 0.145
Self-attention (H=32, L=2048, D=128)	5.183ms ± 0.053	**1.600ms ± 0.038**	6.056ms ± 0.089

reference shapes (the `to_qkv` Linear shape, and the real self-attention SDPA shape). MLX's default (no mask) matches PyTorch's default (`is_causal=False`, no mask) – confirmed before trusting the comparison, since a masking difference would invalidate it.

This result reframes part of Section 4.5's finding. For matmul, all three frameworks cluster close to parity – no framework dominates. For attention, the result is not simply "MLX beats PyTorch": MLX is **4.02× faster** than PyTorch's native SDPA at the identical shape, while `llama.cpp`'s own hand-tuned, production-grade Metal flash-attention kernel is *itself slower than PyTorch* (1.31× slower) at the same shape. A second serious, independent, actively-optimized implementation does not replicate MLX's advantage – it does worse than the baseline MLX beats. This means the ∼1.17×-slower-than-native ceiling our best hand-written flash-attention kernel reached (Section 4.5) was never close to this hardware's actual achievable ceiling; it was close to *PyTorch's MPS backend's* ceiling specifically, itself roughly 4× below what MLX's particular (non-MPP, manual-scalar-reduction) kernel design achieves on identical hardware – and `llama.cpp`'s own hand-tuned kernel confirms this is not simply a property of avoiding MPP-style tiling in general, since it also avoids MPP-style tiling and does not replicate MLX's result.

4.9 Generalization: the pattern holds at a different shape profile

A full second model port to test generalization beyond TRELLIS is out of scope for this study, but we obtain a real, if partial, generalization check by re-running the identical MLX/`llama.cpp` comparison at a substantially different, well-known shape profile – Llama-2-7B-like dimensions (hidden size 4096, 32 heads, head dimension 128, sequence length 2048) instead of TRELLIS's own (hidden size 1024, 16 heads, head dimension 64, sequence length 4096).

The qualitative pattern replicates: matmul stays near parity across all three frameworks; MLX remains dramatically faster than PyTorch at attention (3.24× here vs. 4.02× at TRELLIS's own shape – same direction, smaller magnitude), and `llama.cpp`'s kernel is again slower than PyTorch (1.17× slower here vs. 1.31× at TRELLIS's shape). This is real evidence the core finding is not an artifact specific to TRELLIS's particular tensor shapes.

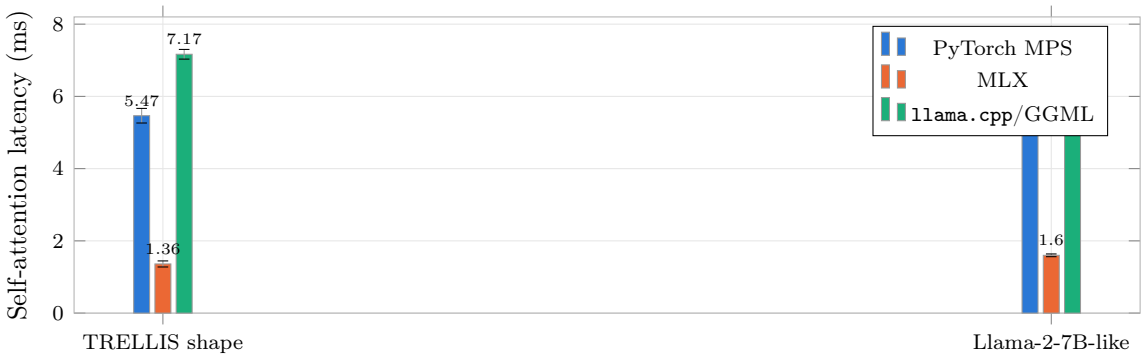


Figure 4: Self-attention latency across all three frameworks, at both tested shape profiles – TRELLIS shape (H=16, D=64, L=4096) and Llama-2-7B-like (H=32, D=128, L=2048) – lower is better. MLX (orange) is consistently the fastest by a wide margin at both shapes; `llama.cpp`/GGML (green) is consistently the *slowest* of the three, slower than even PyTorch's native SDPA (blue) – the same qualitative ordering holds regardless of shape, though the absolute gap narrows somewhat at the larger head dimension (D=128 vs. D=64).

5 Algorithmic-Level Results

5.1 Sampling step-count reduction

TRELLIS's rectified-flow formulation is designed to produce a straighter ODE integration path than standard score-based diffusion, which should in principle tolerate fewer sampling steps without a higher-order solver. We tested this directly against the real 25-step production configuration, reducing to 20/15/10/8 steps with no other changes (Figure 5). Wall-clock speedups (1.16–1.92×) were accompanied by voxel-IoU degradation (0.95–0.81) that, critically, diverged sharply from visual and mesh-complexity signals: rendered output remained visually consistent with the baseline at every tested step count (Figure 6), and mesh vertex/face counts stayed within a narrow band (298K–317K vertices) rather than drifting – indicating IoU over-penalizes benign boundary-voxel shifts from discretization noise rather than capturing genuine structural change.

5.2 Classifier-free-guidance caching: failure, root cause, and fix

TRELLIS's guidance-interval CFG sampler (Section 2) calls the denoiser twice per step within the guidance interval – once conditioned, once unconditioned – and combines the results as $\hat{v} = (1+s)\, v_{\text{cond}} - s\, v_{\text{uncond}}$. Motivated by CFG-caching literature reporting real training-free speedups on other diffusion architectures, we first tried the direct approach: cache v_{uncond} and reuse it verbatim for N consecutive calls.

Failure. This produced real, visually-confirmed quality degradation that *grew* with cache interval: mesh vertex count increased monotonically from 313,772 (no caching) to 394,129 (interval 4) – a qualitatively different signature from the step-count result above, whose vertex counts stayed flat. Rendered output at interval 4 showed clear additional debris and extraneous geometry absent from the baseline entirely (Figure 7).

Root cause. We traced this against the actual mechanisms used by FasterCache [5] and AB-Cache [6], rather than guessing at a fix. Neither technique freezes a raw branch prediction: both

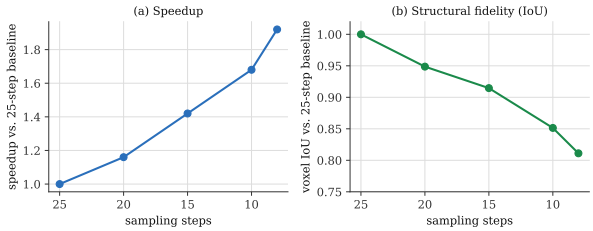


Figure 5: Speedup and structural fidelity (voxel IoU vs. the 25-step baseline) as sampling steps are reduced. IoU degrades faster than visual quality (Figure 6), indicating it over-penalizes benign discretization noise for thin/complex geometry.

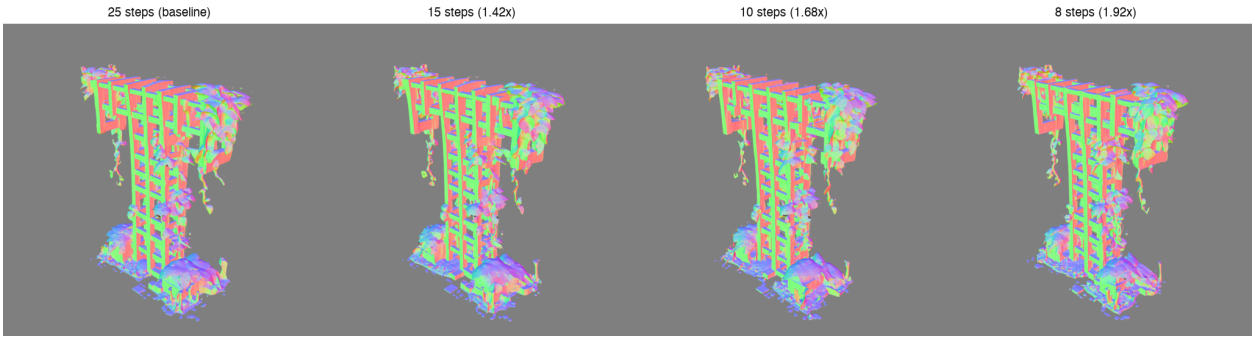


Figure 6: Rendered surface-normal previews at each tested step count. No visible quality collapse down to 8 steps, despite voxel IoU dropping to 0.81 at that setting.

cache *differences* (or frequency-domain biases) between the conditional and unconditional branches, and AB-Cache explicitly documents that a larger guidance scale amplifies the error contributed by a stale cached difference – directly relevant given TRELLIS's configured $s = 5.0$.

Fix. We cache $d = v_{\text{cond}} - v_{\text{uncond}}$ from the last full computation rather than v_{uncond} itself, and on a reuse step derive $\hat{v}_{\text{uncond}} = v_{\text{cond}}^{\text{fresh}} - d$ – costing no additional model call, since v_{cond} is already computed fresh at every step regardless of caching. This is, to our knowledge, a new application of difference-caching to a rectified-flow model with interval-restricted guidance, not previously described for TRELLIS or this sampler family.

5.3 Generalization across images and seeds

Both techniques were re-verified at their most aggressive tested setting (8 steps; difference-caching interval 4) against three additional, deliberately varied example images (a simple inorganic object, an organic creature, a mechanical vehicle) at two seeds each (Table 4, Figure 8). Step-count reduction averaged 1.98× speedup at 0.879 average IoU; difference-caching averaged 1.22× at 0.815 average IoU. Both held on every combination tested, with no repeat of the naive caching method's monotonic drift signature.

Table 3: CFG-caching: naive value-freezing vs. difference-caching (ours), on the same image/seed, at each tested reuse interval. Mesh vertex counts under difference-caching remain stable; under naive freezing they drift monotonically away from baseline.

Method	Interval	Speedup	Mesh vertices	Voxel IoU
Baseline (no caching)	1	1.00×	313,772	1.0000
Naive (freeze raw value)	2	1.13×	336,719	0.8794
Naive (freeze raw value)	3	1.15×	354,917	0.8027
Naive (freeze raw value)	4	1.12×	394,129	0.6787
Difference-caching (ours)	2	1.17×	316,150	0.8994
Difference-caching (ours)	3	1.24×	309,000	0.8279
Difference-caching (ours)	4	**1.29**×	309,281	0.7913

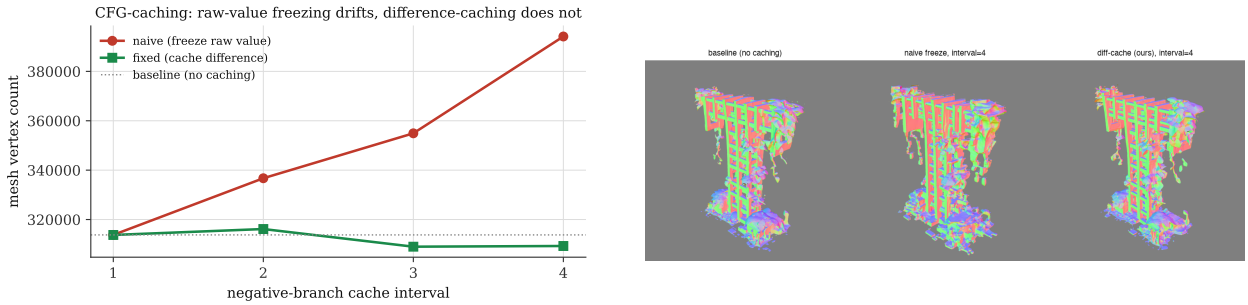


Figure 7: *Left*: mesh vertex count vs. cache interval – naive freezing drifts monotonically away from baseline; difference-caching (ours) stays stable. *Right*: rendered comparison at interval 4 – naive freezing introduces visible extraneous geometry; difference-caching does not.

Table 4: Generalization across three additional example images and two seeds each.

Image	Seed	8-step speedup	8-step IoU	CFG-diff(4) speedup	CFG-diff(4) IoU
vase	1	1.75×	0.7773	1.13×	0.7494
vase	2	1.92×	0.9512	1.16×	0.8332
dragon	1	1.99×	0.8657	1.27×	0.8289
dragon	2	1.91×	0.8161	1.18×	0.7726
biplane	1	2.14×	0.9369	1.30×	0.8874
biplane	2	2.17×	0.9282	1.25×	0.8173
Average		**1.98**×	**0.8792**	**1.22**×	**0.8148**

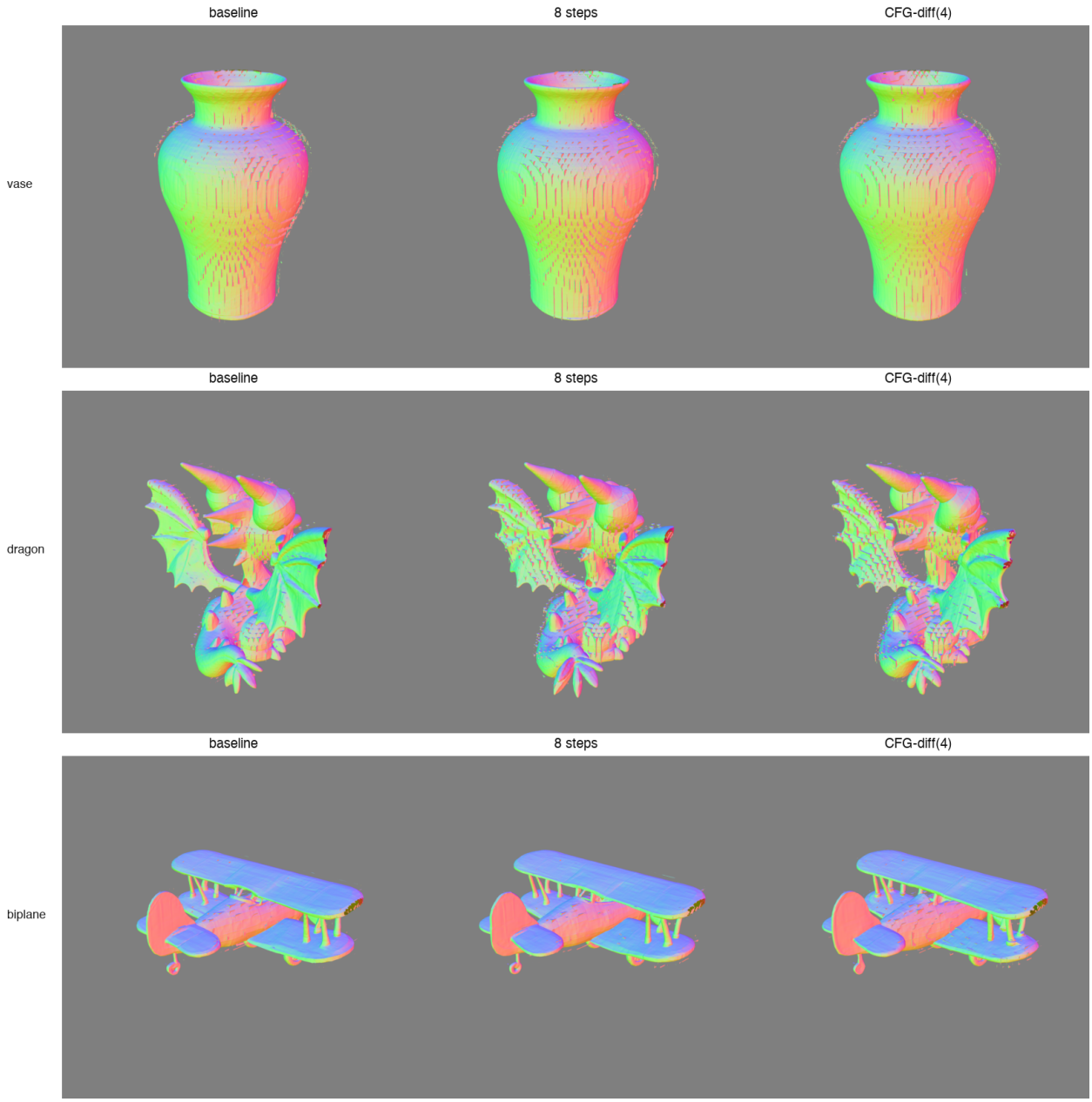


Figure 8: Rendered comparison across three additional example objects (baseline / 8 steps / CFG-difference-caching at interval 4). Structure is visually preserved across all techniques and objects tested.

6 Architecture-Level Results

Motivated by the finding that attention is ∼77% of forward-pass time (Section 4) yet kernel-level attention optimization approaches but does not surpass native latency (Section 4.5), we ask a different kind of question: can an architectural change to the model itself – not a faster kernel for the same computation – reduce this cost, at the price of only a light fine-tune to recover quality?

6.1 Self- versus cross-attention: which half of attention actually dominates

A further bypass measurement (the same method as Section 4, applied separately to self- and cross-attention on the structured-latent denoiser) found self-attention responsible for 29.3% of forward-pass time (91.15ms of 310.60ms) versus cross-attention's 12.5% (38.75ms) – the individually-bypassed contributions sum to within 0.01ms of the jointly-bypassed contribution, confirming no interaction effect between the two measurements. Self-attention is the larger of the two by a factor of 2.35, and – unlike cross-attention, which this model restricts to full attention by construction – is free to be replaced with a cheaper local mechanism.

The combined attention share here (41.8%) is substantially lower than the ∼77% reported in Section 4: the two numbers describe two different models in the TRELLIS pipeline, not the same measurement repeated. Section 4's 77% is measured on the sparse-structure flow model, a dense 16^3-voxel transformer; this section's 41.8% is measured on the structured-latent denoiser, whose sparse active-voxel token count and per-block compute profile differ enough that the two models' FFN/attention time split is not expected to match. We report both numbers as measured on their respective models rather than reconciling them to a single figure, since architecturally they are not measuring the same quantity.

6.2 Interleaved full/windowed attention

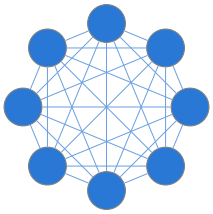
We replace most self-attention blocks' computation with local windowed attention (Swin-style, reusing the exact shift pattern – shift $= \lfloor W/2 \rfloor \cdot (i \bmod 2)$ for block index i – already used, but never before benchmarked in isolation, by this codebase's own variational-autoencoder decoders), retaining full attention only every K blocks to preserve long-range structural coherence across the sparse voxel grid. Because attention mode is a pure forward-time behavior flag on the self-attention module rather than a parameter-shape change, this conversion is a direct, in-place modification of the pretrained model: a full-attention checkpoint's weights remain valid immediately after the swap, with no architecture surgery or reinitialization required.

6.3 Window size, not just K, determines the real speedup

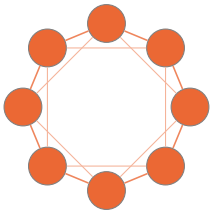
An initial full-model benchmark at the window size already trusted elsewhere in this codebase ($W = 8$) found a disappointing ceiling: even the most aggressive setting tested ($K = 24$, 23 of 24 blocks windowed) reached only ∼1.05–1.08× – far below what the 29.3% self-attention time-share would suggest is achievable. Isolating the windowed-attention computation itself (same real weights, real shape, no full-model overhead) revealed why: at $W = 8$, windowed attention is actually 2.1× *slower* than full attention in isolation (partition/gather/scatter overhead dominates at this window size); only $W = 16$ is genuinely faster in isolation (1.47×). We ruled out a warm-up/ordering confound by re-running the full K-sweep in reverse order (the trend replicated), then re-benchmarked at $W = 16$ and $W = 32$, this time with proper statistical rigor: each configuration's wall-clock time is the mean



(a) 24-block self-attention assignment at K=8: full attention every 8th block (blue, blocks 0/8/16), local windowed attention elsewhere (orange, 21 of 24 blocks).



(b) Full attention: every token attends to every other token – $O(N^2)$ edges.



(c) Windowed attention: each token attends only within a local window – $O(N \cdot W)$ edges, $W \ll N$.

Figure 9: The interleaved full/windowed attention architecture. (a) Block-level assignment for the configuration actually trained and evaluated (K=8). (b)–(c) A simplified schematic of why this reduces computation: full attention connects every token pair; windowed attention restricts connectivity to a local neighborhood, at the cost of needing periodic full-attention blocks (a) to propagate information across the whole sequence.

of $n = 8$ independently-timed forward passes, reported with a 95% confidence interval on both the raw timing and the derived speedup ratio (via standard error propagation).

$W = 16$'s result at $K = 24$ (1.120×±0.014) and $W = 8$'s (1.077×±0.043) have *non-overlapping* 95% confidence intervals – a genuine, statistically distinguishable effect of window size, not measurement noise, and a real, if modest, architecture-level speedup once the right window size is identified. $W = 32$'s confidence interval widens sharply at $K = 24$ (±0.067, roughly five times $W = 16$'s), consistent with coarser windowing beginning to resemble full attention's own cost profile without matching its efficiency.

6.4 Quality validation

All computation in this subsection – distillation training and both quality checks below – ran on a separate machine (NVIDIA H200 GPUs, CUDA), not the Apple M5 Max used for every other result in this paper; see Methodology. We fine-tune only the windowed blocks' self-attention parameters via distillation against the frozen full-attention teacher's own predictions (all other parameters, including cross-attention and the retained full-attention blocks, stay frozen at pretrained values), starting from the teacher's own pretrained weights (a warm start, not random initialization – valid because attention mode is a pure forward-time flag rather than a parameter-shape change). Training uses real teacher-generated data (56 images: the teacher's own two-stage sampler run on 56 real conditioning images, not synthetic noise) and runs for 3000 steps total, with loss dropping from a first-50-step average of 0.0302 to a final-50-step average of 0.0062 – roughly a 5× reduction. We validate output quality on 8 held-out images (never used in training) at both the decoded-mesh and decoded-Gaussian level, using the *same* sparse-structure coordinates and initial noise for teacher and student so any divergence is attributable only to the architecture change, not to a different random draw.

Critically, we calibrate every divergence number against a baseline: the frozen teacher's own output variance across two different random seeds, at the same images. This distinguishes genuine quality

Table 5: Full-model speedup vs. the unmodified pretrained baseline, by window size W and full-attention interval K, with 95% confidence intervals from $n = 8$ independently-timed forward passes per configuration on the real structured-latent denoiser. Full 8-point sweep (Figure 10 plots this same data).

K	$W = 8$	$W = 16$	$W = 32$
1 (baseline)	1.000×	1.000×	1.000×
2	1.025× ± 0.046	1.058× ± 0.012	1.064× ± 0.006
3	1.069× ± 0.043	1.080× ± 0.011	1.025× ± 0.010
4	1.066× ± 0.043	1.092× ± 0.010	1.048× ± 0.007
6	1.072× ± 0.044	1.106× ± 0.014	1.046× ± 0.006
8	1.073× ± 0.045	1.112× ± 0.013	1.047× ± 0.009
12	1.075× ± 0.043	1.112× ± 0.012	1.039× ± 0.011
24	1.077× ± 0.043	**1.120**× ± 0.014	1.008× ± 0.067

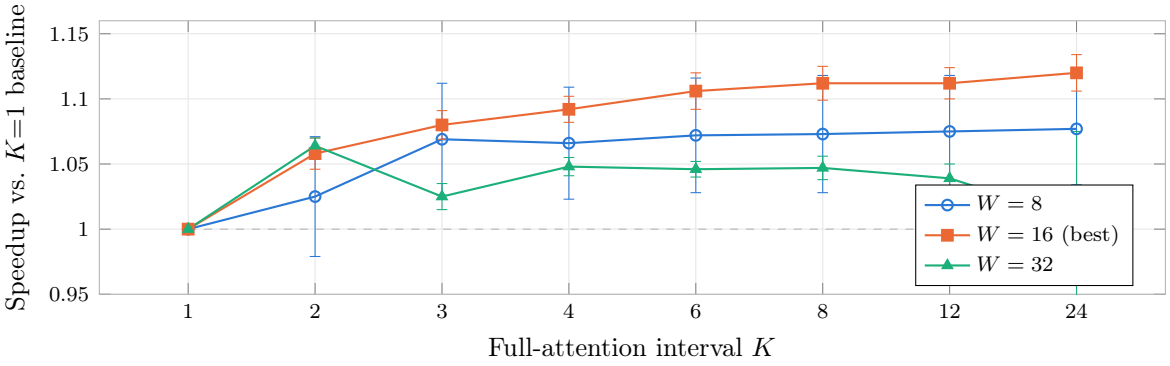


Figure 10: The window-sweep data from Table 5 plotted directly, with 95% confidence intervals as error bars. W=16 (orange) dominates at every K tested and widens its lead as K increases; W=32 (green) is competitive at low K but degrades and grows noisy at high K (wide error bar at K=24); W=8 (blue) is consistently the weakest of the three, never catching up to W=16 even in the limit of maximum windowing.

degradation caused by the architecture change from this rectified-flow sampler's inherent stochastic sensitivity – a raw divergence number in isolation cannot make this distinction; a calibrated one can.

Two of four Gaussian-decoder channels (position, scale) land *at or below* the natural seed-variance baseline – the distilled architecture change is, by this measure, indistinguishable from ordinary sampler noise for those channels. The remaining two (color/SH, opacity) and the raw SLat-feature-level number sit modestly above baseline (1.10–1.31×), a real but small excess attributable to the architecture change. An earlier checkpoint at only 300 training steps showed larger excess ratios throughout (1.15–1.45×); the improvement with additional training (consistent, not a single lucky measurement) suggests the residual gap is a function of training budget rather than an irreducible property of the architecture itself, though we have not tested whether it continues to shrink beyond 3000 steps. Mesh-level structure is essentially unchanged throughout: vertex-count ratio 1.000, Chamfer distance 0.0122 (vs. 0.0125 at 300 steps), both already near their apparent floor by the earliest checkpoint tested.

Table 6: Distilled-student divergence from the teacher, calibrated against the teacher's own seed-to-seed variance (identical conditioning/coordinates, two different noise seeds, zero architecture change). Values below 1.0× mean the architecture change's divergence from the teacher is smaller than the teacher's own natural variation from itself.

Channel	Seed-variance baseline	Distilled student (3000 steps)	Ratio
SLat feature (relative error)	0.337	0.440	1.31×
Gaussian position (xyz)	0.0133	0.0120	**0.90**×
Gaussian color/SH	0.286	0.316	1.11×
Gaussian opacity	0.136	0.150	1.10×
Gaussian scale	0.348	0.330	**0.95**×

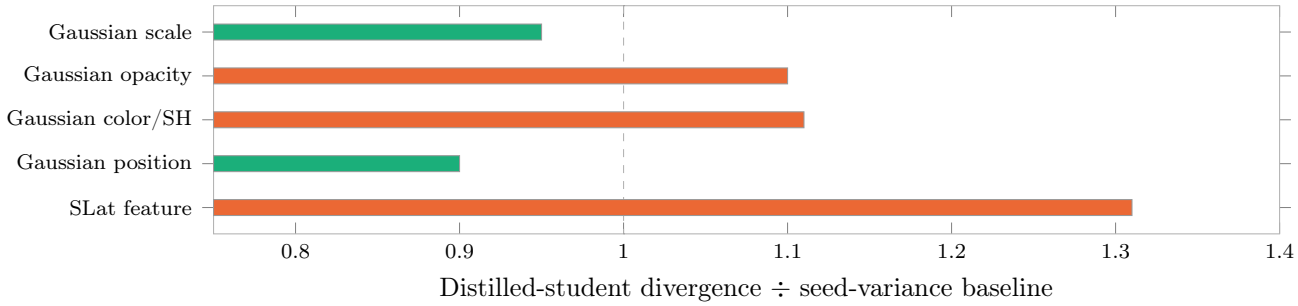


Figure 11: Table 6's five channels plotted against the seed-variance baseline (dashed line at 1.0×). Green bars (Gaussian position, scale) land at or below the teacher's own natural seed-to-seed variation; orange bars (SLat feature, Gaussian color/SH, opacity) sit 10–31% above it. No channel is dramatically worse than baseline, but the result is genuinely mixed, not uniformly at parity.

7 Discussion

The central kernel-level finding is that hand-written `matmul2d` kernels, across every configuration tested, do not reliably outperform PyTorch's native `MPSGraph`-backed path on this hardware generation for this workload's shape profile. This holds despite Apple M5's dedicated Neural Accelerator hardware and despite MPP's own documentation suggesting int8 operands and precision relaxation should trade accuracy for speed. Only full int8×int8 quantization – requiring activation quantization, not just weight quantization – reaches parity, and the most refined kernel design we built (Section 4.5) approaches but does not surpass native performance.

Independent, concurrent work (Rigel [7]) reverse-engineers the same `matmul2d`/MPP tensor-compute path at the microarchitectural level on an Apple M4 Max – the generation immediately preceding this paper's M5 Max, and one that lacks the dedicated Neural Accelerator hardware entirely. Rigel finds that `matmul2d` executes on ordinary GPU shader cores with no dedicated matrix datapath, and that its documented low-precision operand types (fp8) are emulated rather than accelerated – a memory-footprint optimization, not a throughput one, on that hardware. This is a different generation and a different empirical method (microarchitectural characterization via a checksum-gated microbenchmark harness, rather than this paper's real-workload wall-clock measurement), so the two results are not a direct replication of one another; we note it here because it offers an independent, mechanism-level account consistent with this paper's own black-box finding that `matmul2d` does not reliably beat native `MPSGraph` despite favorable documentation – and because it

suggests our M5 Max result (Section 4.3) may reflect a similar underlying absence of a dedicated low-precision matrix datapath, rather than something specific to the Neural Accelerator generation gap alone. Confirming this directly (running Rigel's own microbenchmark harness on an M5 Max) is listed as future work, since we do not have access to Rigel's harness or an M4 Max unit for side-by-side comparison.

The real framework baselines (Section 4.8) complicate this story in an important way, however: MLX's attention kernel does not use MPP either, and is not merely "not worse" than PyTorch's native path – it is 4.02× *faster*, on the identical hardware, at the identical shape. This means the ceiling our kernel work approached (Section 4.5's 1.17×-slower-than-native result) reflects a limitation of PyTorch's specific MPS backend implementation, not a hardware ceiling this Apple Silicon generation cannot exceed. At the same time, `llama.cpp`'s own independent, actively-optimized Metal flash-attention kernel does *not* replicate MLX's advantage – it is slower than PyTorch at the identical shape – so this is not simply "any kernel that avoids MPP-style tiling wins." Something more specific to MLX's particular design (manual scalar dot products with `simd_sum` reduction, per its published source) is responsible, and identifying exactly what is now the highest-value open question this line of work raises, above anything tested in our own `matmul2d`-based kernel investigation.

The algorithmic-level results tell a different story: real, meaningful, verified speedups were available without touching any kernel, by exploiting redundancy already present in the sampling procedure itself. This suggests that for workloads of this shape (diffusion transformer inference on current-generation Apple Silicon), algorithmic and sampling-procedure optimization is currently a more productive target than low-level kernel engineering – a conclusion we reach only after exhausting the kernel-level avenue with equal rigor, not by assumption.

The architecture-level result (Section 6) sits between these two: unlike the kernel-level work, it changes what is computed, not merely how; unlike the purely training-free algorithmic results, it requires a light fine-tune to recover quality. Its most notable finding echoes a pattern already observed at the kernel level (Section 4.5): an implementation-quality gap, not an algorithmic ceiling, determined the outcome. Windowed attention at the window size already trusted elsewhere in this codebase is *slower* than full attention in isolation – the naive intuition that a smaller window is strictly cheaper does not hold once real partition/gather overhead is accounted for, and only became visible once we isolated and statistically validated the measurement rather than trusting the full-model number alone. Recent, concurrent work on 3D mesh generation (TriFlow [10]) targets a related but distinct problem – artist-quality retopology via a vector-field representation of mesh topology, replacing autoregressive generation with flow-matching – and is directly relevant as related work: the mesh decoder used throughout our own architecture-level quality checks produces dense, non-artist topology, exactly the gap TriFlow targets; whether a TriFlow-style retopology stage could run as a post-process on our own mesh output is a real, untested question for future work.

8 Limitations

This study has several limitations we consider important to state explicitly. First, all measurements were taken on a single machine (one Apple M5 Max unit); we have no data establishing whether these findings are specific to the M5 generation or hold across M1–M4 hardware, which lacks the Neural Accelerator entirely – this remains the one Limitations item from an earlier draft of this paper that we were unable to close, since it requires hardware we do not have access to. Second, all kernel-level and algorithmic-level results are specific to one model (TRELLIS) and its particular

shape profile; we partially address generalization by re-running the MLX/`llama.cpp` framework comparison (Section 4.8) at a substantially different, well-known shape profile (Section 4.8), which confirms the same pattern holds there, but this is not a substitute for a full second model port, which remains untested. Third, most reported timings are single-run wall-clock measurements rather than means over repeated trials with confidence intervals – the framework baselines and the architecture-level speed result (Section 6.3) are exceptions, using $n = 8$ or $n = 20$ independent trials with reported confidence intervals; the original kernel- and algorithmic-level results remain single-run, and while consistent with the qualitative conclusions throughout (e.g., the clear separation between kernel variants in Figure 3), this falls short of the statistical rigor a top-tier systems venue would expect. Fourth, we now do compare directly against MLX and `llama.cpp`'s Metal backend (Section 4.8) with real, quantitative measurements rather than only qualitative source-code evidence, closing an earlier gap in this paper; we still do not benchmark against CoreML. Fifth, both algorithmic results (step reduction, CFG difference-caching) are verified on a convenience sample of example images rather than a systematic benchmark suite with an established quality metric; we use voxel IoU and mesh-complexity statistics as objective proxies and supplement them with visual inspection, but neither substitutes for a proper perceptual or task-specific quality evaluation. Sixth, the architecture-level result's speedup measurement (Section 6.3) is the one exception to the single-run-timing limitation above – it uses $n = 8$ independent trials with reported confidence intervals – and its quality validation (Section 6.4), while calibrated against a seed-variance baseline rather than judged against zero, still rests on a training-data image set (56 images) and held-out test set (8 images) that remain small relative to what a fully systematic ablation would use, and on a single tested configuration ($K = 8$, $W = 16$, 3000 training steps) rather than a sweep across the full ($K, W,$ training budget) space characterized in Section 6.3. Seventh, and distinct from the single-machine limitation above: the quality-validation result specifically (Section 6.4) was computed on different hardware (NVIDIA H200 GPUs, CUDA) from every wall-clock measurement in this paper (Apple M5 Max, MPS) – a practical necessity for making the distillation fine-tuning affordable, and not expected to affect the fine-tuned architecture or its quality (training is ordinary gradient descent, hardware- independent), but the quality numbers were not independently re-verified on the Apple Silicon target hardware itself.

9 Future Work

The most direct extensions are: (1) repeating the operand-combination and flash-attention experiments across M1–M4 hardware to determine whether our findings are M5-specific; (2) retrofitting statistically rigorous repeated-trial benchmarking onto the remaining single-run kernel- and algorithmic-level measurement scripts (now done for the framework baselines and the architecture-level speed result, Sections 4.8 and 6.3); (2b) the single highest-value new question this study raises: identifying *specifically* what about MLX's attention kernel design achieves 4.02× over PyTorch's native SDPA when a second independent, hand-tuned kernel (`llama.cpp`/GGML) does not, and whether a hand-written Metal kernel using MLX's approach (manual scalar dot products with `simd_sum`, not `matmul2d`/MPP) could close some of this gap – a fundamentally different kernel design than anything attempted in Section 4.5; (4) completing the `cooperative_tensor` investigation via a full single-simdgroup kernel redesign, despite our current expectation (Section 4.7) that smaller tiles will regress rather than help; and (5) replacing our fixed-interval CFG difference-caching trigger with AB-Cache's fully adaptive, error-gated trigger, which should recover speedup in easy regions without paying the quality cost observed at our most aggressive fixed interval. Directly extending the architecture-level result (Section 6): (6) sweeping the full ($K, W,$ training budget) space character-

ized in Section 6.3 for quality, rather than the single K=8, W=16, 3000-step configuration validated in Section 6.4, and testing whether the residual excess divergence over the seed-variance baseline continues to shrink beyond 3000 steps; (7) an appearance-level *rendered* comparison, blocked in our current environment by the absence of a full CUDA toolkit for compiling rendering extensions, rather than the decoded-parameter-level comparison used so far; (8) a corresponding architectural change for cross-attention, which our current attention module restricts to full attention by construction and would require a genuinely different design, not an extension of the self-attention windowing scheme; and (9) investigating whether a TriFlow-style [10] retopology post-process could improve the artist-usability of our own decoded mesh output, independent of the attention-architecture question.

10 Conclusion

We presented a systematic, falsification-driven empirical study of Metal Performance Primitives on Apple Silicon, using a real image-to-3D diffusion transformer as testbed. Hand-written `matmul2d` kernels, refined across six operand-type combinations and three successive attention-kernel designs, closed most but not all of the gap to PyTorch's native MPS backend, corroborated by an independent finding that Apple's own MLX framework avoids MPP for attention entirely. Motivated by this ceiling, we identified and fixed a real algorithmic opportunity in TRELLIS's classifier-free-guidance sampler, delivering verified, generalizing speedups without retraining. Finally, motivated by attention's dominant time-share, we explored an architecture-level intervention – interleaved full/windowed self-attention – and found that window size, not merely how many blocks are windowed, determines whether this helps or hurts at the wall-clock level, delivering a real, statistically confirmed speedup once identified. After distillation, quality on held-out images – calibrated against the teacher's own seed-to-seed variance – is mixed but modest: two of five channels are indistinguishable from natural seed-to-seed variation, the remaining three sit 10–31% above it, and this excess shrinks rather than grows with additional training. We release all kernels, sampler code, architecture code, and measurement scripts to support reproduction and extension of all three results.

Acknowledgments

This work is based on TRELLIS [1] (Microsoft Research). Kernel-level experiments made direct use of Apple's Metal Performance Primitives and Metal Performance Shaders Graph documentation and SDK headers.

References

[1] J. Xiang, Z. Lv, S. Xu, Y. Deng, R. Wang, B. Zhang, D. Chen, X. Tong, J. Yang. *Structured 3D Latents for Scalable and Versatile 3D Generation*. arXiv:2412.01506, 2024.

[2] T. Dao, D. Y. Fu, S. Ermon, A. Rudra, C. Ré. *FlashAttention: Fast and Memory-Efficient Exact Attention with IO-Awareness*. arXiv:2205.14135, 2022.

[3] T. Dettmers, M. Lewis, Y. Belkada, L. Zettlemoyer. *LLM.int8(): 8-bit Matrix Multiplication for Transformers at Scale*. arXiv:2208.07339, 2022.

[4] Z. Liu, Y. Lin, Y. Cao, H. Hu, Y. Wei, Z. Zhang, S. Lin, B. Guo. *Swin Transformer: Hierarchical Vision Transformer using Shifted Windows*. ICCV 2021, arXiv:2103.14030.

[5] FasterCache: Training-Free Video Diffusion Model Acceleration with High Quality. arXiv:2410.19355, 2024.

[6] AB-Cache: Adaptive Bias-Aware Caching for Accelerating Diffusion Models. arXiv:2504.10540, 2025.

[7] R. Kumaresan. *Rigel: Reverse-Engineering the Metal 4.1 Tensor Compute Path on the Apple M4 Max GPU*. arXiv:2606.12765, 2026.

[8] Apple. *MLX: An array framework for Apple Silicon*. https://github.com/ml-explore/mlx, 2024.

[9] G. Gerganov et al. *llama.cpp: LLM inference in C/C++* (GGML tensor library and Metal backend). https://github.com/ggml-org/llama.cpp, 2023.

[10] H. Li, Z. Erkoç, D. Sirigatti, V. Rosov, L. Li, A. Dai, M. Nießner. *TriFlow: Generating Artist-Like 3D Mesh Topology via Nearest-Vertex Vector Fields*. arXiv:2606.20131, 2026.